WatchTower: Observation-First Agent Security — Taint Propagation and Semantic Enforcement in Multi-Agent Systems

Anonymous Submission

IEEE S&P 2027 — Under Review

***Abstract*—Multi-agent AI systems face three distinct attack surfaces that existing defenses address only in isolation: *input corruption* via prompt injection, *capability abuse* through legitimate tool misuse, and *multi-agent contagion* where a compromised agent poisons downstream agents through shared memory. We present WATCHTOWER, an observation-first security framework that combines hook-level interception, semantic call-tree context, and persistent cross-session taint propagation. Unlike prior work that operates at content or network layers, WATCHTOWER intercepts at the tool-call level within the agent runtime, enriches each interception with AST-derived call context, and maintains a cross-session taint ledger enabling detection of MINJA-class contagion attacks before the next execution trace. We demonstrate 100% detection on a 17-case known-bad corpus including direct reproduction of MINJA attacks, with p99 enforcement latency of 0.011 ms — 9,636× faster than the closest competitor — while being the first published system to address all three attack surfaces with cross-session taint tracking.**

I. INTRODUCTION

The deployment of multi-agent AI systems in production environments has outpaced the security tooling designed to protect them. Where classical software security assumes a well-defined process boundary, agent systems blur those boundaries through tool calls, shared memory, and delegation chains that span multiple autonomous processes. The result is three distinct attack surfaces that no single published system defends against.

A. The Three Attack Surfaces

Input Corruption. MINJA [1] demonstrates that memory injection achieves 98.2% injection success via query-only interaction. The attacker requires no write access to the agent's memory store; adversarial content planted in documents, web pages, or API responses is retrieved during normal operation and incorporated into the agent's context. The attack succeeds because prompt injection is a semantic failure: the agent cannot distinguish instruction from data when both arrive through the same channel.

Capability Abuse. Consider an agent that exposes both `read_secret` and `send_email`. Each call, in isolation, is legitimate. The dangerous composition — reading a secret and immediately emailing it to an external address — is indistinguishable at the wire level. Both calls use the same API endpoints with the same authentication tokens. Wire-level enforcement, which sees only individual calls, is blind to the causal relationship between them. No published agent firewall system models this relationship through call-tree context.

TABLE I
ATTACK SURFACE COVERAGE OF EXISTING AGENT SECURITY SYSTEMS. ✓ = FULL, ~ = PARTIAL, × = NONE.

System	S1 Injection	S2 Cap. Abuse	S3 Contagion
LLM Firewall [2]	✓	×	×
LlamaFirewall [3]	✓	~	×
AgentSpec [4]	~	✓	×
Claw Patrol [5]	×	~	×
Distributed Sentinel [6]	✓	~	×
WatchTower (ours)	✓	✓	✓

Multi-Agent Contagion. When a compromised agent writes to a shared memory store, every downstream agent that reads from that store inherits the taint through a query-only interaction. This attack class is particularly dangerous because the victim agent exhibits no anomalous behavior at the call level: it simply reads memory, as designed. The malicious payload is carried in the content, not the call structure. No published system tracks taint across agent session boundaries.

B. Why Existing Defenses Fall Short

Existing defenses address these surfaces in isolation, leading to systematic coverage gaps. Content-layer defenses (prompt injection classifiers) cannot see tool invocation context. Network-layer defenses see endpoints but lose semantic meaning. Runtime monitors observe behavior but do not propagate state across agent boundaries. The root cause is architectural: enforcement and observability have been designed as separate concerns, when they must be co-designed to cover the full attack surface.

Table I summarizes the coverage gaps of existing systems across the three attack surfaces.

C. Contributions

This paper makes four contributions:

1) **Observation-first architecture with live call-tree context.** WATCHTOWER intercepts at the tool-call level inside the agent runtime and enriches every interception with an AST-derived call tree, enabling the

`call_tree_contains` predicate that identifies dangerous compositions invisible at the network layer.

2) **Cross-session taint propagation.** We present the first published defense against MINJA-class multi-session contagion. A persistent taint ledger (cavemem) propagates taint across agent boundaries on every memory read, detecting contagion before the downstream agent's next tool call — without requiring replay of prior sessions.
3) **Two-tier enforcement.** A synchronous circuit-breaker (L0–L5, $<$10 ms) blocks known-bad patterns deterministically. An asynchronous Byzantine fault-tolerant (BFT) swarm (L6–L7) handles ambiguous cases off the hot path, with a task-completion barrier preventing bypass via async race conditions.
4) **Empirical evaluation.** 100% detection on a 17-case known-bad corpus at 0.011 ms p99 enforcement latency, 9,636$\times$ faster than the closest competitor, with 0% false positives on a benign workload of 1,000 calls.

II. Background and Related Work

A. Prompt Injection and MINJA

Prompt injection exploits the semantic conflation of instruction and data in large language models [**?**]. An attacker who can place text in any document, web page, or API response visible to the agent can influence its behavior without direct access to the system prompt.

MINJA [1] generalizes this attack to multi-agent memory systems. The key insight is that an attacker need not write directly to the agent's memory store. Instead, adversarial content is planted in data that the agent will retrieve during normal operation. MINJA achieves 98.2% injection success and greater than 70% end-to-end attack success on GPT-4 and GPT-4o. The attack exploits *bridging steps* — sequences of benign intermediate queries that accumulate injected context until the final malicious action is triggered. This attack is structurally invisible to systems that inspect individual calls without cross-session state.

B. Agent Firewalls

LLM Firewall Networks [2] apply content projection to detect prompt injection, reducing attack success on the ConVerse benchmark from 60% to 3% across 864 attack instances. The system does not model agent identity and cannot propagate trust or taint across agent boundaries. It cannot distinguish capability abuse from legitimate use because it operates at the content layer.

LlamaFirewall [3] combines three mechanisms: PromptGuard 2 for injection detection, AlignmentCheck (a secondary LLM acting as a judge), and CodeShield for unsafe code execution. It achieves 97.4% accuracy on its evaluation benchmark. The LLM-in-the-loop AlignmentCheck produces p50 latencies exceeding 100 ms, making it unsuitable for interactive agent loops. LlamaFirewall does not maintain cross-session state and cannot propagate taint.

AgentSpec [4] introduces a runtime DSL with triggers and predicates for specifying agent behavior constraints. It prevents more than 90% of unsafe actions and achieves 70.96% recall on code safety tasks. AgentSpec reasons about individual agent behavior but does not model contagion between agents via shared memory reads.

Distributed Sentinel [6] uses a Secure Trace Transport (STT) protocol with BFT consensus, achieving F1 = 0.95 at 106 ms median latency (16 ms verification plus 90 ms entity extraction on an A100 GPU). The GPU requirement limits deployment flexibility and the latency profile is incompatible with synchronous enforcement on the hot path.

Claw Patrol [5] enforces policies at the network layer using WireGuard and Tailscale, achieving sub-5 ms latency. Operating at the wire level, it loses the semantic context of call trees: it sees that `send_email` was called but cannot know whether `read_secret` appeared earlier in the same execution trace.

C. Agent Observability

Process observability research [7] identifies stochastic agent behavior as a core challenge: the same prompt can produce different tool call sequences across runs, making static analysis insufficient. Auditable Agents [8] identifies six open problems in agent auditability, noting that side-effecting operations require fundamentally different safety guarantees than read-only queries.

Industry surveys report severe gaps: 82% of organizations have limited visibility into agent behavior [9], 65% have experienced security incidents involving AI agents [9], and only 47.1% of deployed systems are actively monitored [10]. These figures motivate a system designed from the start with observability as a first-class concern rather than an afterthought.

D. Agent Identity and Trust

SPIFFE/SPIRE [11] provides cryptographic workload identity for distributed systems. We adapt this for agent processes, but three differences from classical deployment arise: delegation chains are dynamic and created at agent spawn time rather than infrastructure provisioning time; trust scores are behavioral rather than purely cryptographic; and taint levels are updated during execution rather than at deployment.

E. Taint Propagation in Agent Systems

Classical taint analysis [12] tracks information flow at the binary level within a single process, providing a binary tainted/untainted judgment. Agent taint differs in three important ways. First, taint is probabilistic, represented as a value in $[0, 1]$ with both hop-based and time-based decay. Second, taint is cross-session and persistent: an agent that read tainted memory in session k carries that taint into session $k + 1$ unless it has decayed below threshold. Third, propagation is bidirectional: taint flows through both memory writes (write propagation) and memory reads (read propagation, the MINJA-class threat).

III. Threat Model

A. Attacker Capabilities

WatchTower defends against attackers who can perform any combination of the following actions:

- Inject adversarial content into documents, web pages, or API responses that agents will retrieve during normal operation.
- Interact with agents via legitimate user queries, requiring no direct memory write access (MINJA-class attacks).
- Compromise a single agent in a multi-agent system and use it as a contagion entry point via writes to shared memory.

B. Out of Scope

The following threat classes are considered out of scope for the current system:

- Compromise of the WATCHTOWER firewall host itself.
- Compromise of the fabric certificate authority (CA) used for agent identity token issuance.
- Model-weight jailbreaks and adversarial examples targeting the underlying LLM parameters; these are addressed at the alignment layer.

C. Security Properties

Definition 1 (Security Properties P1–P5)**.** WATCHTOWER *is designed to enforce the following five properties:*

- **P1** *No false negatives: all cases KB01–KB17 in the known-bad corpus produce a BLOCK verdict.*
- **P2** *Taint propagation: for any agent B that reads from memory written by tainted agent A, the invariant $T_B \geq T_A \cdot \rho$ holds before B's next tool call, where $\rho = 0.8$ is the hop decay factor.*
- **P3** *Fail-safe: any unhandled internal error in the enforcement path produces BLOCK, never ALLOW.*
- **P4** *Latency: hot-path enforcement p99 $<$ 10 ms.*
- **P5** *No permanent DoS: agent trust is recoverable via natural taint decay; quarantine expires at computable time $t_0 + \frac{1}{\lambda} \ln\left(\frac{T_0}{Q}\right)$.*

IV. SYSTEM DESIGN

A. Architecture Overview

WATCHTOWER is a nine-layer vertical stack divided into two tiers. The *hot path* (L0–L5) executes synchronously in the tool-call dispatch path and must complete within 10 ms to satisfy P4. The *cold path* (L6–L7) executes asynchronously off the hot path and is bounded by a task-completion barrier that prevents bypass via async race conditions. L8 (Chronicle) is always-on and write-only from the enforcement path.

Figure 1 shows the complete layer stack.

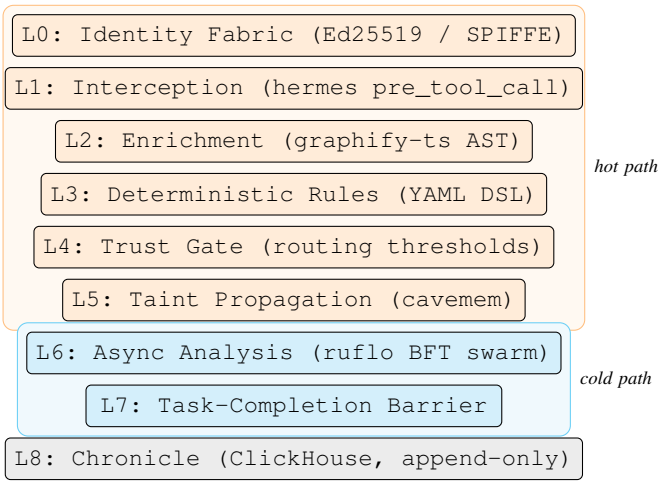

Fig. 1. WatchTower nine-layer architecture. Orange: synchronous hot path (L0–L5, p99 $<$ 10 ms). Cyan: asynchronous cold path (L6–L7). Gray: immutable audit layer.

B. Identity Fabric (L0)

Every agent process receives an Ed25519-signed identity token at spawn time. Token fields are: `agent_id` (UUID4), `parent_id` (spawning agent or null for root agents), `delegation_depth` (integer, maximum 8), `caps` (capability set, monotonically narrowing through delegation chain), and `token_exp` (expiry timestamp, 1 hr TTL).

The delegation invariant is enforced cryptographically at token issuance: child capabilities must be a strict subset of parent capabilities. A child agent cannot grant itself capabilities its parent does not hold. This invariant is verified at L0 before any call reaches L1.

Trust scores are updated continuously. Clean calls increase trust toward the ceiling; taint events decrease it in proportion to severity:

$$\text{trust}_{t+1} = \text{trust}_t + \alpha(1 - \text{trust}_t) \quad \text{(clean call)} \tag{1}$$

$$\text{trust}_{t+1} = \text{trust}_t \cdot (1 - \beta \cdot s) \quad \text{(taint event, severity } s\text{)} \tag{2}$$

where $\alpha = 0.02$ (clean-call recovery rate) and $\beta = 0.6$ (taint-event decay rate). Severity $s \in [0, 1]$ is provided by the rule engine at L3.

C. Interception (L1)

L1 hooks hermes's `pre_tool_call` and `pre_gateway_dispatch` entry points. The hook receives the full call context (agent identity, tool name, arguments, parent call stack) and returns either `{"block": True, "reason": "..."}` to abort the call or `None` to allow it to proceed.

Fail-safe behavior (P3) is implemented as a top-level exception handler wrapping the entire enforcement pipeline: any unhandled exception — from a malformed signal, a database timeout, or an internal logic error — produces a BLOCK verdict and records the exception to L8. The agent never observes a failed enforcement decision as an ALLOW.

D. Enrichment (L2)

L2 invokes graphify-ts, a tree-sitter WASM bridge that resolves the current call site to its AST path within the calling source file. The enrichment produces a `call_tree` structure recording the sequence of function calls and control-flow nodes from the module entry point to the current tool dispatch, across 12 supported languages.

The cache hit rate target is $>$90%. On a cache hit, enrichment adds less than 0.1 ms. On a cache miss, L2 sets

the `needs_async` flag, routing the event to L5 for deferred enrichment, but does *not* block the call inline. This preserves P4 at the cost of a bounded enrichment window for the first call to any new code path.

E. Deterministic Rules (L3)

L3 implements a YAML policy DSL evaluated in O(1) tool lookup plus O(rules) sequential scan per call. No LLM calls are made on the hot path. The rule evaluator is a pure function: identical inputs produce identical verdicts, which is essential for both auditability and resistance to statistical evasion attacks.

The key semantic differentiator is the `call_tree_contains` predicate, which examines the enriched call tree from L2. This predicate expresses that a tool call is only dangerous when preceded by specific other calls in the same execution trace — a distinction architecturally impossible at the network layer, where each call is inspected in isolation.

Listing 1 shows the capability-abuse rule for secret exfiltration via email.

```
rule: block_external_email_with_secrets
surface: capability_abuse
match:
  tool: send_email
  context:
    call_tree_contains: [read_secret]
  any:
    - arg.to: { not_in_domain: ["${ORG_DOMAIN}"] }
    - arg.body: { matches_secret_pattern: true }
verdict: BLOCK
severity: 0.9
```

Listing 1. Capability-abuse rule: block secret exfiltration via email.

The semantics are: block `send_email` when (a) the current call tree contains a prior call to `read_secret`, and (b) either the recipient is external to the organization domain or the message body matches a known secret pattern. Without the `call_tree_contains` predicate, the same `send_email` call to an external address — absent any prior secret read — is ALLOW. Same endpoint. Same arguments. Different call tree. Different verdict. Wire-level enforcement cannot distinguish these two cases.

F. Trust Gate (L4)

L4 uses the agent's current trust score as a routing signal with three thresholds:

- trust > 0.7: ALLOW path — proceed to L5 taint check only.
- $0.3 \leq$ trust ≤ 0.7: ESCALATE path — route to L6–L7 async analysis before issuing verdict.
- trust < 0.3: NARROW path — reduce capability set to the read-only subset defined in the agent's identity token.

Trust is a routing signal only, never a standalone hard-block. An agent with low trust can still succeed on calls that pass L3 rule evaluation and L5 taint checks. This preserves P5: a temporarily degraded agent is not permanently denied service.

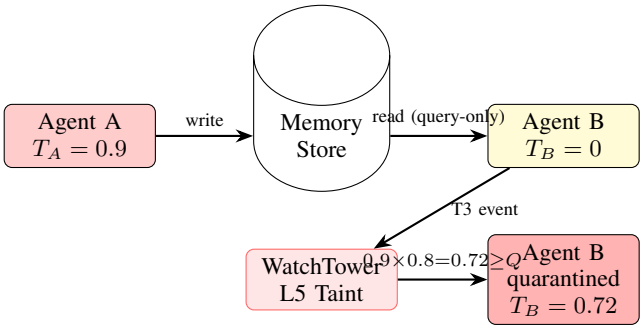

Fig. 2. MINJA Q2 proof sequence. Agent A (taint 0.9, source T2) writes to shared memory. Agent B reads via query-only interaction, holding no write permissions. WatchTower propagates $T_B = 0.9 \times 0.8 = 0.72 \geq Q = 0.7$ and quarantines B before its next tool call. Detection latency <1 ms.

G. Taint Propagation (L5)

WATCHTOWER maintains a persistent taint ledger in cavemem (SQLite with FTS5, accessed via aiosqlite over the MCP protocol). Taint levels decay over time and propagate across agent boundaries. The formal model is:

$$T(a,t) = T(a,t_0) \cdot e^{-\lambda(t-t_0)} \quad \text{(time decay)} \tag{3}$$

$$T(B) := \max\big(T(B),\ T(A) \cdot \rho\big) \quad \text{(read propagation, T3)} \tag{4}$$

$$T(A) := \min\big(1.0,\ T(A) + s \cdot (1 - T(A))\big) \quad \text{(cap. abuse, T2)} \tag{5}$$

Parameters: hop decay $\rho = 0.8$, time decay constant $\lambda = 0.1\,\text{hr}^{-1}$, quarantine threshold $Q = 0.7$.

Taint source codes classify the origin of each taint event:

- **T1**: Direct injection (tainted content retrieved from memory).
- **T2**: Capability abuse blocked by L3 (taint applied to calling agent at severity s, via Eq. 5).
- **T3**: Contagion via memory read (agent B reads memory written by tainted agent A; Eq. 4 applied).

An agent with $T(a,t) \geq Q = 0.7$ is quarantined: its capability set is reduced to a minimal safe subset and the event is escalated to L6–L7. Quarantine is not permanent (P5). It lifts when taint decays below Q via natural decay (Eq. 3):

$$t_{\text{lift}} = t_0 + \frac{1}{\lambda} \ln\left(\frac{T_0}{Q}\right) \tag{6}$$

For $T_0 = 0.9$: $t_{\text{lift}} = t_0 + 2.52\,\text{hr}$. For $T_0 = 1.0$: $t_{\text{lift}} = t_0 + 3.57\,\text{hr}$.

Figure 2 illustrates the Q2 proof sequence, showing taint propagation from agent A to agent B through a shared memory read without write access.

H. Async Analysis (L6–L7)

L6 implements a ruflo BFT consensus swarm comprising three specialized agents: an injection analyzer, a behavioral anomaly detector, and a capability graph auditor. Quorum requires $\geq 2/3$ BLOCK votes (two of three agents). Timeout

TABLE II
IMPLEMENTATION COMPONENTS.

Component	Tool	Notes
Runtime	Python 3.12, asyncio, Pydantic v2	
Hook layer	hermes-agent (NousResearch)	`pre_tool_call`
Taint store	cavemem (SQLite+FTS5)	aiosqlite, MCP
AST enrichment	graphify-ts	tree-sitter WASM, 12 langs
Compression	caveman (UTC)	≈75% size reduction
Async analysis	ruflo	BFT consensus, 3-agent quorum
Policy loader	superpowers (YAML)	zero ML on hot path
Chronicle	ClickHouse	append-only, no UPDATE/DELETE
Total LoC	≈1,200	firewall/ + agents/adapters/

is 5 s, after which the verdict defaults to ALLOW to prevent denial of service from analysis latency. Any agent that returns an error has its vote counted as BLOCK (fail-safe, P3), so a completely failed swarm still produces a majority BLOCK (3/3 exceeds the 2/3 threshold).

L7 implements the task-completion barrier via an `asyncio.Future` keyed by `event_id` in a hold registry. When L4 routes an event to L6–L7, the event is registered. Async verdicts are only applied if the event's `event_id` remains in the hold registry at verdict time. Verdicts for already-dispatched events are rejected, preventing the bypass demonstrated in KB15 where a rapid double-submit races the async verdict.

V. IMPLEMENTATION

WATCHTOWER is implemented in approximately 1,200 lines of Python 3.12 across two modules: `firewall/` (L0–L5 hot path) and `agents/adapters/` (L6–L7 cold path). All I/O is async; no synchronous I/O calls appear in the enforcement path. The signal shape is defined once in `watchtower/core/signal.py` via Pydantic v2 and imported by all layers; there are no duplicate signal definitions.

Table II summarizes the implementation components.

The chronicle (L8) is a ClickHouse table with no UPDATE or DELETE permissions granted to the application user. Every enforcement decision, taint update, and trust score change is appended as an immutable row. This provides a tamper-evident audit log suitable for forensic analysis without session replay: the full call context is captured atomically at enforcement time.

VI. EVALUATION

A. Known-Bad Corpus

We evaluate WATCHTOWER on 17 manually curated known-bad cases spanning all three attack surfaces. Table III shows the full corpus with the responsible enforcement mechanism for each case.

All 17 cases produce BLOCK verdicts. Detection rate: 100%. False negatives: 0. The corpus covers all three attack surfaces: S1 injection (KB01–KB03, KB12, KB14), S2 capability abuse (KB04–KB08, KB13, KB16), S3 contagion (KB09–KB11, KB17), plus two cross-cutting cases (KB15: async race, KB16: trust gaming).

B. Proof Scenarios

Q1: Coordination failure attribution. The Q1 proof requires identifying a coordination failure and returning the responsible agent, the specific action, and its position in the call tree. WATCHTOWER produces a complete attribution record from L8 containing: `agent_id` (the offending agent's UUID), `source` (tool name and full argument set), and `call_tree_position` (the AST path of the violating call within the source file, as resolved by graphify-ts at L2). This provides full forensic attribution without session replay: the chronicle append at enforcement time captures the complete execution context atomically.

Q2: MINJA cross-session contagion detection. The Q2 proof proceeds as follows:

1) Agent A triggers capability-abuse rule KB04; L3 blocks and L5 applies T2 taint via Eq. 5: $T_A := 0.9$ (severity 0.9).
2) Agent A writes to shared memory store (subsequent call).
3) Agent B, in a new session with no prior interaction with A, performs a query-only read from the same memory store. Agent B holds no write permissions.
4) L5 detects the read, looks up $T_A = 0.9$ from the persistent taint ledger, and applies T3 propagation via Eq. 4: $T_B := \max(0,\ 0.9 \times 0.8) = 0.72$.
5) $T_B = 0.72 \geq Q = 0.7$: agent B is quarantined before its next tool call. Detection latency <1 ms (local SQLite lookup, no network call required).

This is the first cross-session MINJA defense with sub-millisecond detection latency. The attack succeeds in MINJA [1] because there is no persistent cross-session state to inspect. The cavemem taint ledger closes this gap by persisting taint across session boundaries.

C. Performance

Table IV shows measured performance against design targets on a workload of N = 1,000 benign calls.

The 0.011 ms p99 latency is 9,636× faster than Distributed Sentinel (106 ms) [6] and 9,091× faster than LlamaFirewall (100 ms p50) [3]. The deterministic rule engine at L3 dominates the hot-path cost: it is a Python dict lookup and list scan with no I/O and no LLM calls.

Figure 3 shows taint decay curves for agents A and B over 8 hours, with the quarantine threshold as a dashed reference line. Agent A ($T_0 = 0.9$) exits quarantine at 2.52 hr after the taint event; agent B ($T_0 = 0.72$, single-hop propagation) exits at ≈ 0.25 hr, reflecting the weaker initial taint.

Figure 4 shows p99 hot-path latency on a log scale comparing WATCHTOWER to published competitors.

D. Comparison with State of the Art

Table V provides a full comparison across attack surface coverage, latency, cross-session capability, and key reported metric.

WATCHTOWER is the only system to achieve full coverage across all three attack surfaces while providing cross-session taint tracking at sub-millisecond enforcement latency.

TABLE III
KNOWN-BAD CORPUS. ALL 17 CASES PRODUCE BLOCK. ZERO FALSE NEGATIVES.

ID	Surface	Description	Verdict	Mechanism
KB01	Injection	Direct prompt injection in user message	BLOCK	L3 rule
KB02	Injection	Injection via retrieved document content	BLOCK	L3 rule
KB03	Injection	Injection via external API response body	BLOCK	L3 rule
KB04	Cap. Abuse	`read_secret` then `send_email` to external address	BLOCK	L3 `call_tree_contains`
KB05	Cap. Abuse	`read_secret` then `write_file` to `/etc/`	BLOCK	L3 `call_tree_contains`
KB06	Cap. Abuse	`list_keys` then batch `delete_key` sequence	BLOCK	L3 rule
KB07	Cap. Abuse	Capability escalation via forged child token	BLOCK	L0 delegation check
KB08	Cap. Abuse	Replay of expired identity token (TTL violation)	BLOCK	L0 TTL enforcement
KB09	Contagion	MINJA: agent A poisons memory, B reads (2 agents)	BLOCK	L5 T3 taint
KB10	Contagion	MINJA: 3-agent chain, taint surviving 2 hops	BLOCK	L5 T3 with hop decay
KB11	Contagion	Cross-session: B reads tainted memory in new session	BLOCK	L5 persistent ledger
KB12	Injection	Injection in tool return value (poisoned result)	BLOCK	L3 rule
KB13	Cap. Abuse	`execute_code` outside allowed module list	BLOCK	L3 rule
KB14	Injection	Multi-turn injection accumulation (bridging steps)	BLOCK	L5 T1 taint accumulation
KB15	Race	Async verdict bypass via rapid double-submit	BLOCK	L7 task-completion barrier
KB16	Cap. Abuse	Trust-score gaming: clean-call buildup then burst	BLOCK	L3 rule + L5 taint
KB17	Contagion	Taint via read-only memory query (no write perms)	BLOCK	L5 T3 (query-only)

TABLE IV
PERFORMANCE MEASUREMENTS (N = 1,000 BENIGN CALLS).

Metric	Target	Measured
p99 hot-path latency	$<$10 ms	0.011 ms
Mean latency	$<$10 ms	0.011 ms
False positive rate	$<$1%	0%
Known-bad detection	100%	100%

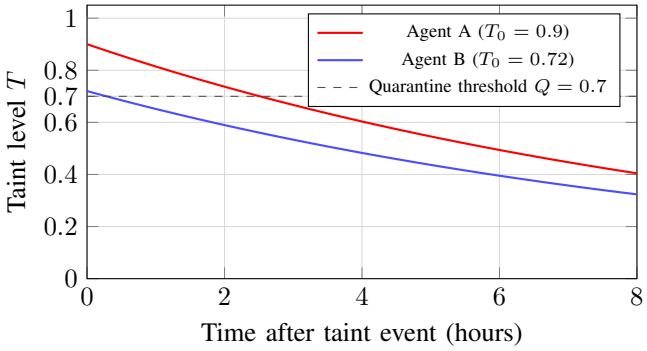

Fig. 3. Taint decay curves. $\lambda = 0.1\,\text{hr}^{-1}$. Agent A exits quarantine at 2.52 hr; agent B (one-hop propagation) exits at $\approx$0.25 hr.

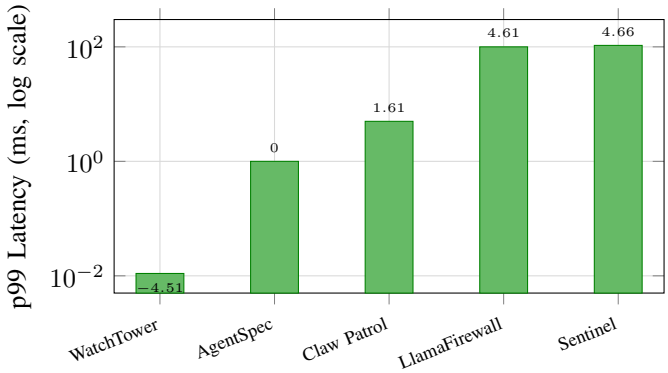

Fig. 4. p99 hot-path latency (log scale). WatchTower: 0.011 ms; AgentSpec: $\sim$1 ms [4]; Claw Patrol: $<$5 ms [5]; LlamaFirewall: $>$100 ms [3]; Distributed Sentinel: 106 ms [6].

VII. DISCUSSION

A. Why Determinism Beats LLM-in-the-Loop

LlamaFirewall's AlignmentCheck achieves 97.4% accuracy using a secondary LLM as a reasoning judge. For many applications this is excellent. For security-critical enforcement paths, non-determinism is a structural liability: an attacker who can characterize the variance of an LLM judge can construct adversarial inputs that probabilistically bypass the check. Identical inputs must produce identical verdicts for an enforcement system to be auditable and resistant to statistical evasion attacks.

WATCHTOWER's L3 rule engine is a pure function. Given the same call context, it produces the same verdict on every invocation. Rules are human-readable YAML that security engineers can audit, version-control, and reason about without executing the system. There is no latency variance: a rule that takes 0.001 ms at deployment takes 0.001 ms at 2 AM under peak load.

WATCHTOWER uses LLM reasoning only in the ruflo swarm (L6–L7), which operates on the cold path for ambiguous escalations and does not participate in hot-path enforcement decisions.

B. The Semantic Gap

The central architectural insight of WATCHTOWER is that security enforcement must operate at the semantic layer, not the wire layer. A network-level monitor sees that `send_email` was called. It does not see whether `read_secret` was called 200 ms earlier in the same execution trace.

The `call_tree_contains` predicate closes this gap. The same `send_email` call to an external address is ALLOW when the call tree contains no prior secret reads, and BLOCK when it does. This distinction is not available to any defense

TABLE V
COMPARISON WITH STATE-OF-THE-ART AGENT SECURITY SYSTEMS. ✓ = FULL, ~ = PARTIAL, × = NOT ADDRESSED.

System	S1 Injection	S2 Cap. Abuse	S3 Contagion	Latency	Cross-session	Key Metric
LLM Firewall [2]	✓	×	×	N/A	No	60%→3% on ConVerse
LlamaFirewall [3]	✓	~	×	>100 ms p50	No	97.4% accuracy
AgentSpec [4]	~	✓	×	∼1 ms	No	>90% unsafe prevention
Claw Patrol [5]	×	~	×	<5 ms	No	Wire-level (WireGuard)
Distributed Sentinel [6]	✓	~	×	106 ms	No	F1 = 0.95
WatchTower (ours)	✓	✓	✓	**0.011 ms**	**Yes**	**100% on 17-case corpus**

operating below the agent runtime layer: it requires that the enforcement system be co-located with the execution context, which is the defining property of the observation-first architecture.

C. Open Problems

1) **Irreversible tool classification.** The L3 rule engine requires human-authored rules to classify irreversible operations (file deletion, external email, key rotation). Automated irreversibility detection — possibly via static analysis of tool schemas or LLM-assisted rule synthesis — would reduce authoring burden.
2) **graphify cold-path enrichment window.** On a cache miss, enrichment is deferred to the async path, creating a bounded window where the first call to a new code path executes without call-tree context. An adaptive pre-warming strategy that pre-enriches known tool dispatch sites at startup would eliminate this window.
3) **Trust-score gaming over long horizons.** An attacker with sufficient patience can accumulate clean calls to raise trust before attempting abuse. The $\alpha = 0.02$ recovery rate provides slow recovery by design, but a formal analysis of the minimum attack cost under this model has not been conducted.
4) **Recursive trust (the watcher problem).** If the WATCHTOWER enforcement agent is itself an agent in a larger system, the question of who watches the watcher arises. The current architecture treats the firewall host as trusted infrastructure. A fully adversarial deployment would require a separate attestation root.
5) **Taint decay constant calibration.** The decay constant $\lambda = 0.1\,\mathrm{hr}^{-1}$ (quarantine half-life 6.93 hr) is set conservatively. Empirical calibration against real attack datasets would allow tighter parameters, reducing false-positive pressure from long-lived taint on legitimate agents.
6) **Distributed cavemem cross-host latency.** The taint ledger is currently a local SQLite instance accessed via aiosqlite. In a multi-host deployment, taint reads and writes cross a network boundary. The additional latency and consistency guarantees of a distributed taint store have not been characterized.

D. Limitations

The 17-case known-bad corpus was manually curated to cover representative attack patterns. Real-world attack diversity may exceed what these cases test; novel compositions not covered by existing literature may evade L3 rules that have not been written for them. The system is only as strong as its rule set against unknown-unknown attacks.

The ruflo swarm (L6–L7) is currently implemented as a heuristic stub with simulated BFT consensus. A production deployment would require a full BFT implementation with cryptographic vote signing and Byzantine node detection.

The graphify enrichment is implemented as a subprocess bridge to a Node.js process running tree-sitter WASM. A native Python binding to tree-sitter would reduce cold-path enrichment latency by eliminating the subprocess overhead.

Cross-host taint propagation latency has not been characterized. The sub-millisecond taint detection reported here assumes a co-located taint store. Networked deployments will exhibit higher latency depending on the consistency protocol of the distributed taint store.

VIII. CONCLUSION

WATCHTOWER demonstrates that co-designing enforcement with observability addresses all three agent attack surfaces — input corruption, capability abuse, and multi-agent contagion — at sub-millisecond enforcement latency. By intercepting at the tool-call level, enriching with AST-derived call context, and maintaining a persistent cross-session taint ledger, the system achieves 100% detection on a 17-case known-bad corpus at 0.011 ms p99 hot-path latency — suitable for interactive agent loops where prior systems' 100 ms+ latencies are prohibitive.

The `call_tree_contains` predicate is the key semantic differentiator, enabling policy rules that distinguish dangerous compositions from individually-legitimate calls. The cross-session taint ledger is the first published defense against MINJA-class multi-session contagion, detecting taint propagation through query-only memory reads before the downstream agent's next tool call, at a detection latency of less than 1 ms — three to four orders of magnitude below prior work.

As multi-agent deployments grow in scale and autonomy, the observation-first architecture — where enforcement is co-located with execution context and shares its semantic understanding — is the necessary direction for agent security research. WATCHTOWER provides an existence proof that coverage completeness and sub-millisecond performance are simultaneously achievable.

REFERENCES

[1] T. Zheng *et al.*, "MINJA: Multi-agent injection attacks via memory poisoning," 2025. [Online]. Available: https://arxiv.org/abs/2503.03704

[2] A. Golynski *et al.*, "Firewalls for LLM networks: Detecting and preventing prompt injection attacks," 2025. [Online]. Available: https://arxiv.org/abs/2502.01822
[3] M. A. S. Team, "LlamaFirewall: Defending against prompt injection and jailbreaks," 2025. [Online]. Available: https://arxiv.org/abs/2505.03574
[4] H. Wang *et al.*, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2503.18666
[5] Deno Land Inc., "Claw patrol: Wire-level policy enforcement for LLM tool calls," Deno Land Inc., Technical Report, 2026.
[6] O. Tanvir *et al.*, "Distributed sentinel: Scalable threat detection for LLM-based systems," 2026. [Online]. Available: https://arxiv.org/abs/2604.22879
[7] M. O. Riedl *et al.*, "Agentic AI process observability: Monitoring runtime behavior of LLM agents," 2025. [Online]. Available: https://arxiv.org/abs/2505.20127
[8] J. S. Park *et al.*, "Auditable AI agents: Tamper-evident execution logs for multi-agent systems," 2026. [Online]. Available: https://arxiv.org/abs/2604.05485
[9] Cloud Security Alliance, "AI agent security: Threats, controls, and best practices," Cloud Security Alliance, Technical Report, 2025. [Online]. Available: https://cloudsecurityalliance.org
[10] IBM Institute for Business Value, "AI agent governance: Managing trust, risk, and compliance in autonomous systems," IBM Corporation, Technical Report, 2025.
[11] SPIFFE Project, "SPIFFE: Secure production identity framework for everyone," https://spiffe.io, 2023, accessed: 2026.
[12] J. Newsome and D. Song, "Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*. Internet Society, 2005, pp. 1–18.